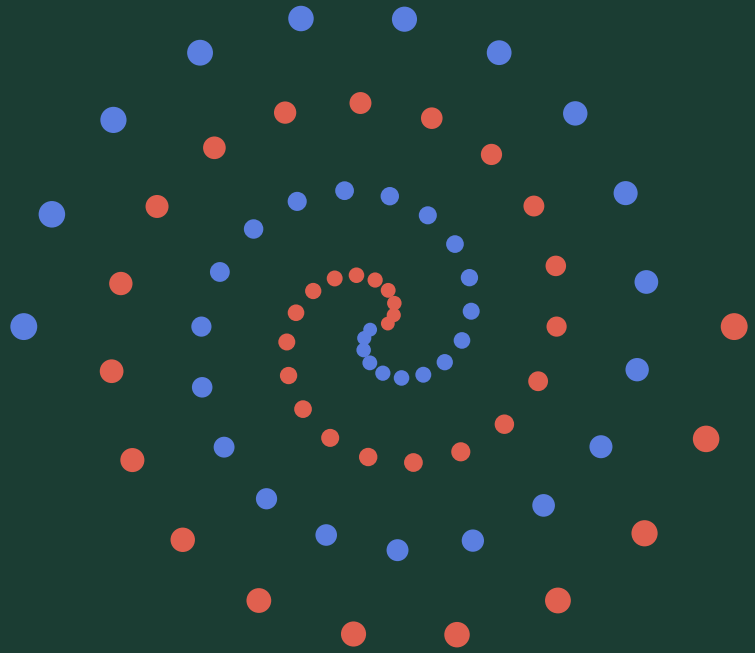
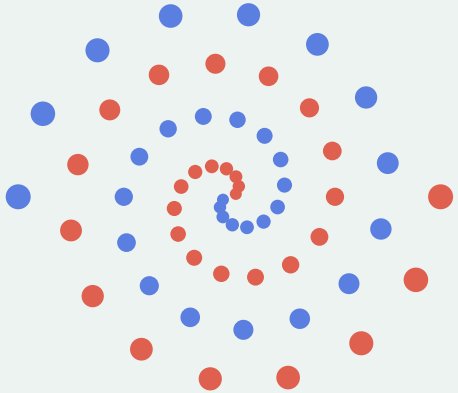
Neural networks through matrix operations

Affine maps, backpropagation, PCA and SVD



UE25MA242A | Mathematical Foundation for AI & Data Science

What are we doing?



Two spirals, two colors

1

The puzzle

Sort red dots from blue dots when they wind around each other

2

The catch

A single straight line cannot split them cleanly

3

The trick

Stack simple steps with one twist (ReLU), so it learns a winding boundary by trial and error

Finally, we use PCA and SVD to look inside and see what it learned

Why this project?

A typical project

- One tool at a time
- For example, PCA on faces or SVD image compression
- Run once and read the answer

This project

- Three course topics working together
- A neural network built from scratch in pure NumPy, no PyTorch or TensorFlow
- Watch it learn, then look inside with PCA and SVD

The question: why can't a straight line separate two spirals? Adding ReLU, a simple switch that bends the line, lifts test accuracy from about 59% to 90%.

Built on three course projects

Matrix transformations

Each layer is an affine map: multiply, then shift

In our work: Each layer is a weight matrix

PCA

Principal directions of centered data

In our work: PCA of hidden activations

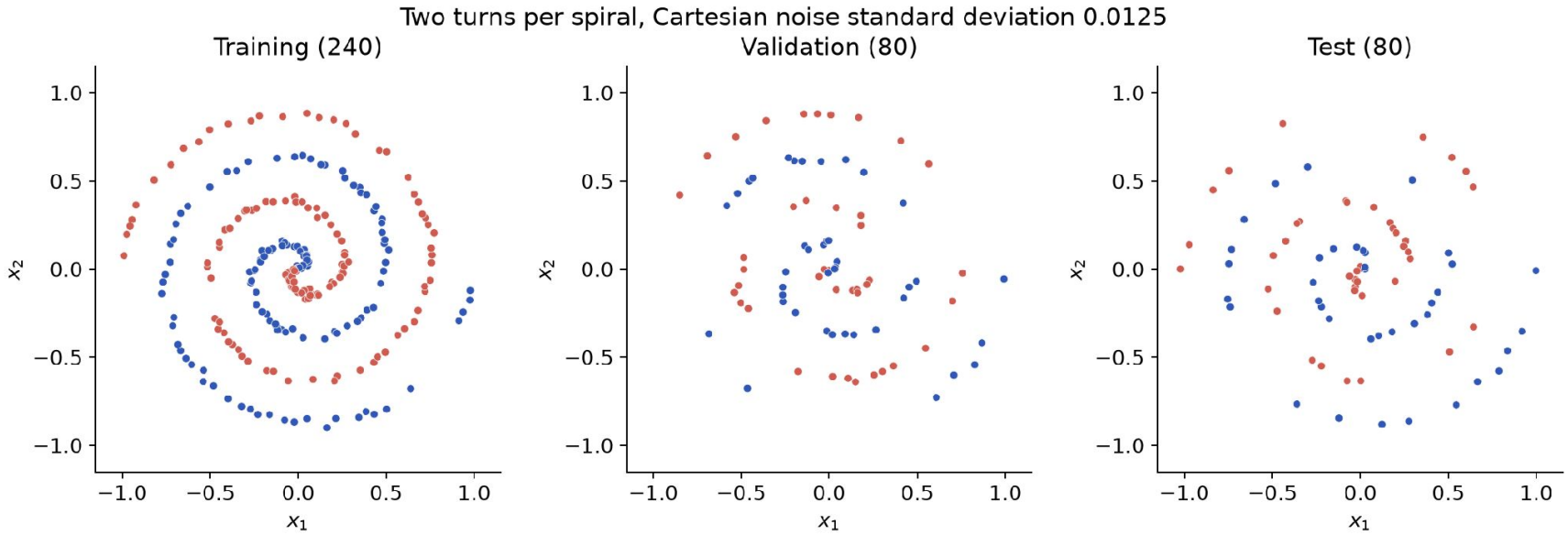
SVD compression

Rank-k approximation of a matrix

In our work: Rank truncation of W2 (16 x 16)

An inspired extension of the supplied problem sheet, not a literal listed problem

Two spirals, three disjoint splits



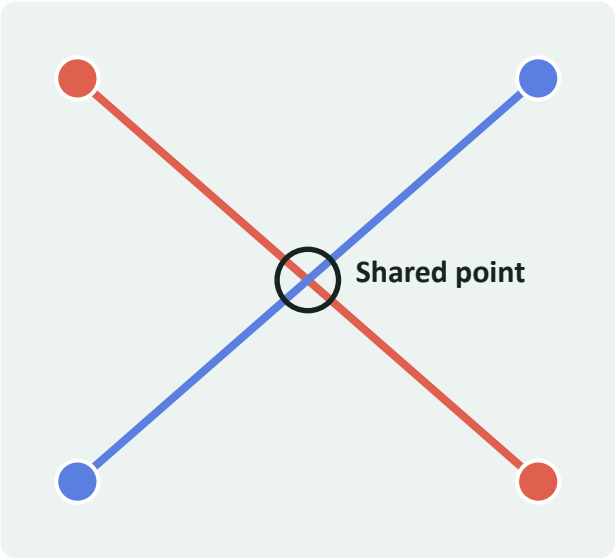
400 points, two turns per arm

Noise standard deviation 0.0125

240 / 80 / 80 train / val / test

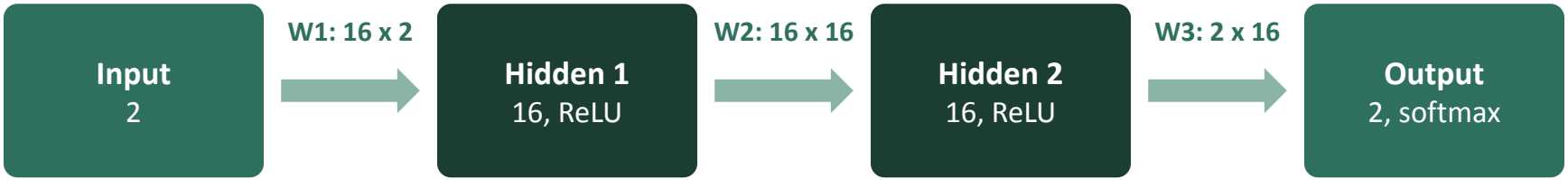
A line cannot separate every point

1. A straight-line rule (an affine map) draws a straight boundary
2. Two same-class segments cross each other across classes
3. Our code computes this crossing point exactly, and it lies in both convex hulls
4. One point cannot take opposite signs, so perfect separation is mathematically impossible



Impossible perfect separation does not mean a 50% accuracy ceiling

A 2-16-16-2 network with 354 parameters



Weights are adjustable dials, biases are shifts, and ReLU is a switch that bends the line.

Map	Weights	Biases	Parameters
Input to hidden 1	16 x 2	16 x 1	48
Hidden 1 to hidden 2	16 x 16	16 x 1	272
Hidden 2 to output	2 x 16	2 x 1	34
Total	**320 entries**	**34 entries**	**354**

How data flows through the network

Multiply

Weights are dials: how much each input counts

Add

A bias is a shift added to the result

Bend (ReLU)

A switch: keeps positives, zeroes negatives

Decide

Softmax turns scores into percentages

Why ReLU matters

Without it, all the layers collapse into one straight line. With it, the line can bend and wind.

How the network learns

1

Measure the error

Compare the prediction with the true class

2

Send it backwards

Each layer learns how much it contributed

3

Work out the nudge

Every weight gets a small correction

4

Update and repeat

12,000 rounds of small steps

Every step is plain matrix multiplication; transposes carry the error backwards

Double-checking our math

35

weights tested one by one

~0

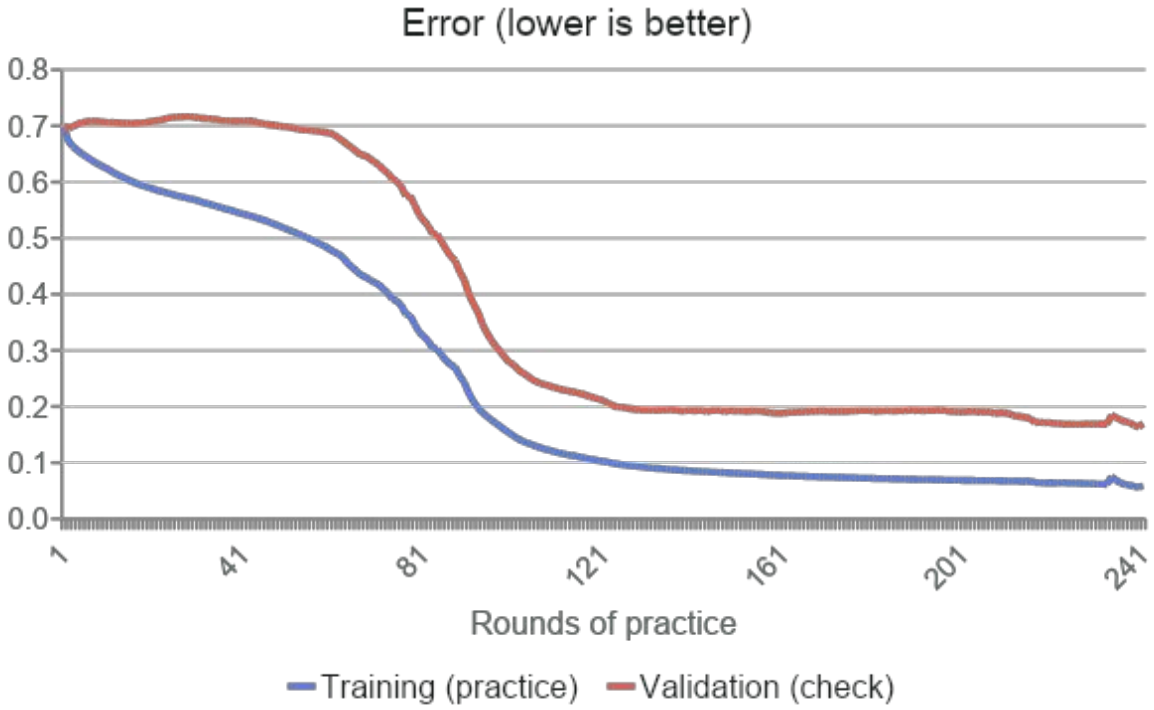
largest difference found (9.89e-11)

7

separate math checks passed

We tested our math a second, independent way, and the answers match.

The error falls as the network practices



12,000
rounds of practice

Blue vs red
practice data vs separate check data

11,950
round we kept: lowest error on check data

Deep network: 90% test accuracy

Model	Params	Train	Val	Test	Epoch
Linear	6	50.00%	45.00%	58.75%	4500
Shallow	82	57.50%	51.25%	58.75%	50
Deep	**354**	**97.08%**	**93.75%**	**90.00%**	**11950**

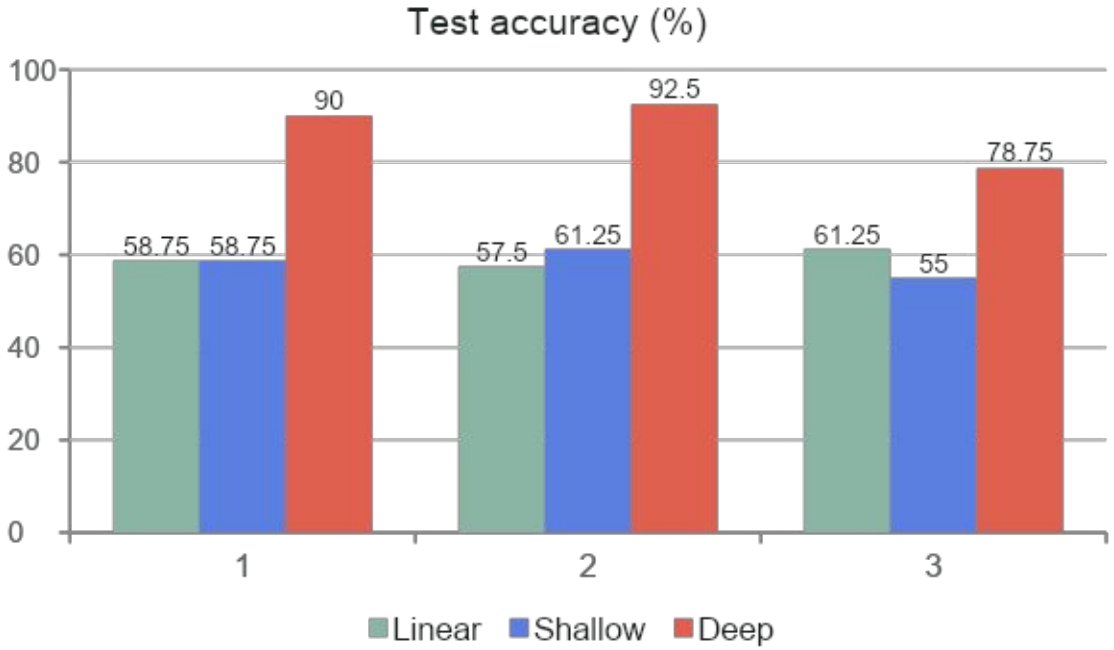
72/80

test predictions correct

Plausible range: about 81% to 95%

The goal is to show that ReLU breaks the straight-line limit, not to chase a leaderboard score. Same split and budget for every model; not a capacity-matched comparison.

Deep results depend on the seed



Linear mean SD 1.91 pp

59.17%

Shallow mean SD 3.15 pp

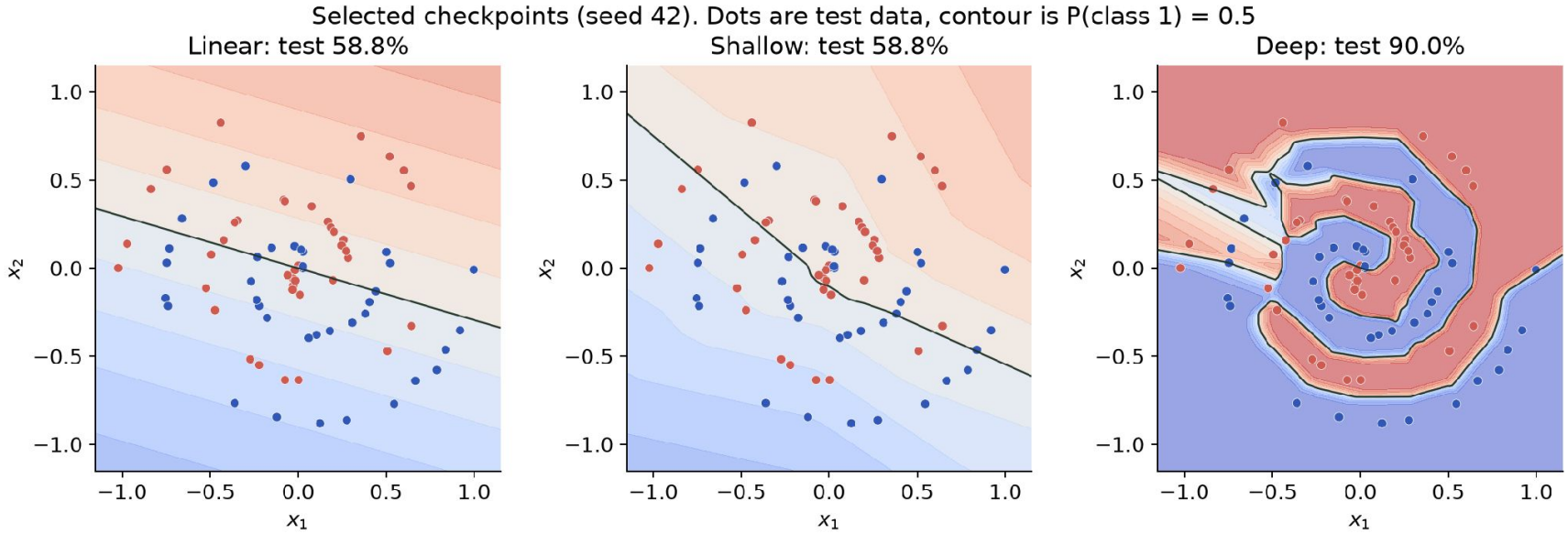
58.33%

Deep mean SD 7.32 pp

87.08%

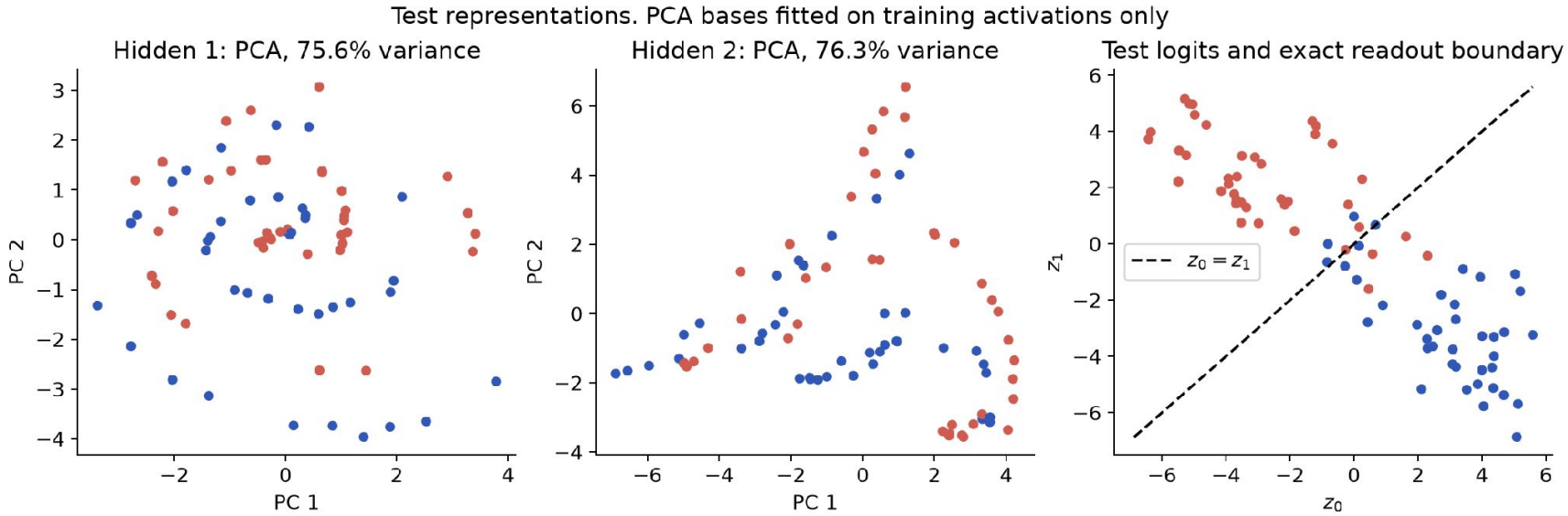
All three seeds shown, not just the best: 78.75% to 92.5%

The learned decision boundaries



Dots are test points; the black line is where the network is 50/50. Linear and shallow stay near 59%.

Looking inside with PCA



PCA squeezes 16 numbers per point into a 2D picture, so some detail is lost.

What the weights do: stretch and squeeze

1.2

Layer 1

stretches evenly

198

Layer 2

stretches very unevenly

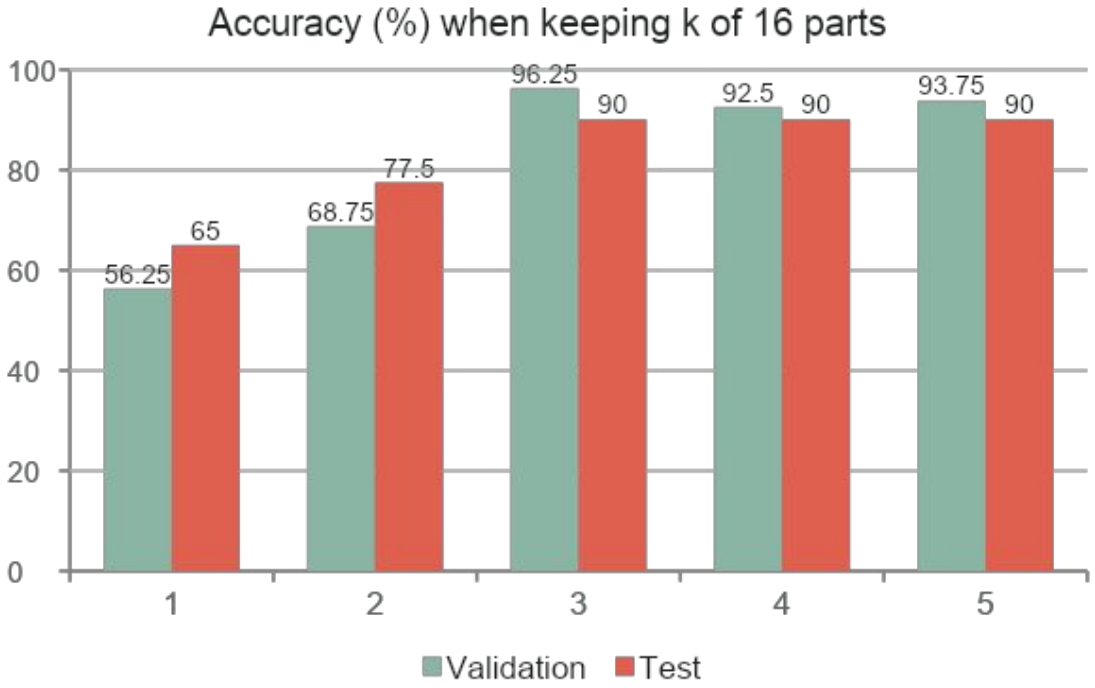
16 to 2

Layer 3

squeezes 16 numbers into 2 scores

SVD measures how much a weight matrix stretches its input. The big number is the stretch ratio: biggest stretch divided by smallest.

Shrinking a layer: half is enough



The idea

Keep only a layer's most important parts and see if it still works.

The result

Half the parts (8 of 16) still reach 90%. Too few (2) drops to 65%.

Conclusions and limits

1. A ReLU network learns the winding spiral boundary: 90% test versus about 59% for the baselines
2. Our hand-built matrix gradients pass an independent check
3. PCA describes activations and SVD describes weights: different objects

Limits

- One split, three seeds
- Small synthetic data, no hyperparameter search
- Shallow failure here does not mean shallow networks must fail

github.com/Annadata-Aniketh/Neural-Net-Maths